

Amrita School of Computing, Amritapuri
Department of Computer Science and Engineering
S6 BTech CSE
23CSE314 Compiler Design(3-0-2-4)
Lab Sheet-0(Practice Lab Sheet)

CO1	Apply theoretical concepts for the analysis of program structure and identify syntax errors.	BTL2
-----	--	------

Design and Implementation of a Simple Lexical Analyzer in Java

Objective

Design and implement a simple program in Java to divide a given input program into a sequence of tokens such as keywords, identifiers, operators, delimiters, and numeric literals. This must be implemented using a progressive two-stage approach and modular programming techniques.

a. Hard-coded Token Identification

In the first stage, implement token identification using basic conditional logic.

- Identify tokens using if–else or switch–case statements
- Hard-code keywords, operators, and delimiters
- Implement separate methods to identify each token type

b. Table-Driven Token Identification

In the second stage, modify the program developed in Stage 1 to use lookup tables.

- Store keywords using HashSet or HashMap
- Store operators using HashMap
- Replace all hard-coded checks with table lookups
- Maintain a symbol table to store identifiers

1. Given the declaration statements

```
int a;  
int b;
```

Identify keywords, identifiers, and delimiters.

Expected Output:

```
<int, KEYWORD>  
<a, IDENTIFIER>
```

2. Given the Assignment Statements

```
int a = 10;
int b = a + 20;
```

Identify keywords, identifiers, operators, numbers, and delimiters.

Expected Output:

```
<int, KEYWORD>
<a, IDENTIFIER>
<=, OPERATOR>
<10, NUMBER>
<;, DELIMITER>
```

```
desktop\Sem 6 Labsheets\Compiler Design\" && javac labsheet0.java && java
labsheetF-8 -Dstderr.encoding=UTF-8
F-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF
-8
<int, KEYWORD>
<a, IDENTIFIER>
<=, OPERATOR>
<10, NUMBER>
<;, DELIMITER>
<int, KEYWORD>
<b, IDENTIFIER>
<=, OPERATOR>
<a, IDENTIFIER>
<+, OPERATOR>
<20, NUMBER>
<;, DELIMITER>
```

3. Given the Control Structures

```
if (a > b) {
    a = b;
}
```

```

java labsheet0_b
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
<if, KEYWORD>
<(, DELIMITER>
<a, IDENTIFIER>
<>, OPERATOR>
<b, IDENTIFIER>
<), DELIMITER>
<{, DELIMITER>
<a, IDENTIFIER>
<=, OPERATOR>
<b, IDENTIFIER>
<;, DELIMITER>
<}, DELIMITER>

Symbol Table: [a, b]

```

Identify keywords, identifiers, operators, numbers, and delimiters.

4. Identification of Invalid Tokens

Given:

```

int 2a;
float @value;

```

Identify valid and invalid tokens.

```

Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
<int, KEYWORD>
<2a, INVALID>
<;, DELIMITER>
<float, KEYWORD>
<@value, INVALID>
<;, DELIMITER>

Symbol Table: []

```

Skeleton Code for HashMap-Based Implementation

```
import java.util.*;
public class SimpleLexer {

    static HashSet<String> keywords = new HashSet<>();
    static HashMap<String, String> operators = new
HashMap<>();
    static HashSet<Character> delimiters = new HashSet<>();
    static HashSet<String> symbolTable = new HashSet<>();

    public static void main(String[] args) {

        initializeTables();

        String input =
            "int a = 10;\n" +
            "int b = a + 20;";

        tokenize(input);
    }

    static void initializeTables() { keywords.add("int");
keywords.add("float"); keywords.add("if"); keywords.add("else");

        operators.put("+", "ARITHMETIC_OP");
operators.put("-", "ARITHMETIC_OP");
operators.put("*", "ARITHMETIC_OP");
operators.put("/", "ARITHMETIC_OP");
operators.put "=", "ASSIGN_OP");
operators.put(">", "RELATIONAL_OP");

        delimiters.add(';');
delimiters.add('(');
delimiters.add(')');
delimiters.add('{');
delimiters.add('}');
    }

    static void tokenize(String input) {

        StringTokenizer st =
            new StringTokenizer(input, " ;(){}=+-*/>\n",
true);

        while (st.hasMoreTokens()) {
            String token = st.nextToken().trim();

            if (token.isEmpty())
                continue;

            if (isKeyword(token)) {
                System.out.println("<" + token + ", KEYWORD>");
            }
            else if (isOperator(token)) {
                System.out.println("<" + token + ", OPERATOR>");
            }
        }
    }
}
```

```

    }
    else if (isDelimiter(token)) {
        System.out.println("<" + token + ", DELIMITER>");
    }
    else if (isNumber(token)) {
        System.out.println("<" + token + ", NUMBER>");
    }
    else if (isIdentifier(token)) {
        symbolTable.add(token);
        System.out.println("<" + token + ",
IDENTIFIER>");
    }
    else {
        System.out.println("<" + token + ", INVALID>");
    }
}

static boolean isKeyword(String token) {
    return false;
}

static boolean isOperator(String token) {
    return false;
}

static boolean isDelimiter(String token) {
    return false;
}

static boolean isNumber(String token) {
    return false;
}

static boolean isIdentifier(String token) {
    return false;
}

```